Category Theory Applied to Functional Programming

Systems Engineering Undergraduate Project

Juan Pedro Villa Isaza
Supervisor: Andrés Sicard Ramírez

EAFIT University

2014

Contents

Introduction
Summary
Overview
Categories
A Category for Haskell
Functors
Functors in Haskell
Natural Transformations
Natural Transformations in Haskell
Conclusions
Future Work
Bibliography

Introduction

- Category theory is a branch of mathematics developed in the 1940s.
- It is a convenient conceptual framework based on:
  - Categories
  - Functors
  - Natural transformations

Introduction

- Category theory occupies a central position in:
  - Computer science
    - Functional programming
- Several concepts of functional programming languages are based on concepts of category theory:
  - Functors
  - Parametrically polymorphic functions
  - Monads
  - Algebraic data types
  - ...

Introduction

Summary of the Project

We examine functional programming through category theory.

Introduction

Summary of the Project

Aim

- To study some of the applications of category theory to functional programming.
- In the context of:
  - Haskell
    - Functional programming language
  - Agda
    - Dependently typed functional programming language
    - Proof assistant

Introduction

Summary of the Project

Objectives

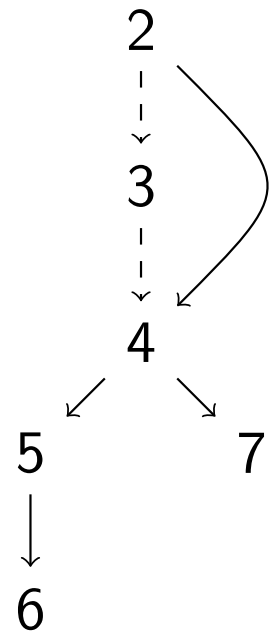
- To describe and explain the concepts of category theory needed for conceptualizing and better understanding:
  1. Functors
  2. Parametrically polymorphic functions
  3. Monads
  4. Algebraic data types

Introduction

Overview of the Project

Chapters

- Chapter 2 (Categories)
- Chapter 3 (Constructions)
- Chapter 4 (Functors)
- Chapter 5 (Natural Transformations)
- Chapter 6 (Monads and Kleisli Triples)
- Chapter 7 (Algebras and Initial Algebras)



Introduction

Overview of the Project

Chapters

- Chapter 2 (Categories)

Category theory

- Categories
- Commutative diagrams

Functional programming

- A category for Haskell
- A category for Agda

Aim and objectives

- All

Introduction

Overview of the Project

Chapters

- Chapter 3 (Constructions)

Category theory

- Isomorphisms
- Initial and terminal objects
- Products and coproducts

Functional programming

- Empty and unit types in Haskell and Agda
- Products and sums in Haskell and Agda

Aim and objectives

- All

Introduction

Overview of the Project

Chapters

- Chapter 4 (Functors)

Category theory

- Functors
- Endofunctors

Functional programming

- Functors in Haskell
- Functors in Agda

Aim and objectives

- Objective 1 (Functors)

Introduction

Overview of the Project

Chapters

- Chapter 5 (Natural Transformations)

Category theory

- Natural transformations

Functional programming

- Parametrically polymorphic functions in Haskell

Aim and objectives

- Objective 2 (Parametrically polymorphic functions)

Introduction

Overview of the Project

Chapters

- Chapter 6 (Monads and Kleisli Triples)

Category theory

- Monads
- Kleisli triples
- Monads = Kleisli triples

Functional programming

- Monads in Haskell
- Monads in Agda

Aim and objectives

- Objective 3 (Monads)

Introduction

Overview of the Project

Chapters

- Chapter 7 (Algebras and Initial Algebras)

Category theory

- Algebras and initial algebras over endofunctors

Functional programming

- Algebraic data types and folds in Haskell

Aim and objectives

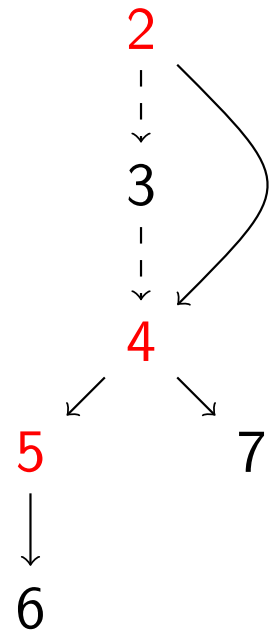
- Objective 4 (Algebraic data types)

Introduction

Overview of the Presentation

Chapters

- Chapter 2 (Categories)
- Chapter 3 (Constructions)
- Chapter 4 (Functors)
- Chapter 5 (Natural Transformations)
- Chapter 6 (Monads and Kleisli Triples)
- Chapter 7 (Algebras and Initial Algebras)



Categories

Definition (Category)

A category $\mathcal{C}$ consists of:

- Objects a, b, c, ...
- Morphisms $f : a \to b$, $g : b \to c$, ...
- Domain and codomain objects $a = \mathrm{dom}(f)$ and $b = \mathrm{cod}(f)$ for $f : a \to b$, ...
- Identity morphisms $\mathrm{id}_a : a \to a$, ...
- Composite morphisms $g \circ f : a \to c$ for $f : a \to b$ and $g : b \to c$, ...

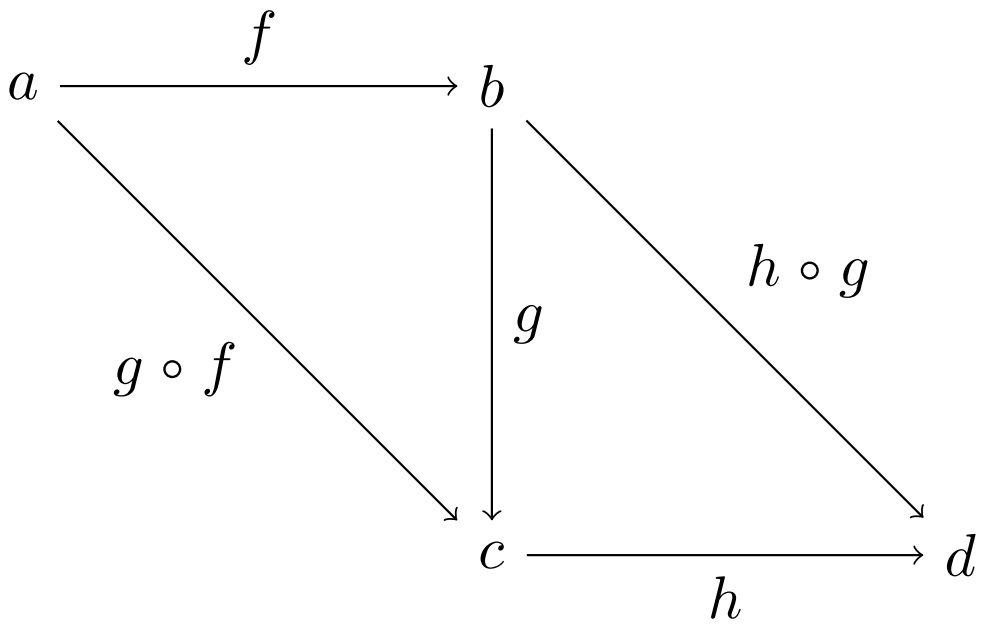
Such that:

- Associativity: $h \circ (g \circ f) = (h \circ g) \circ f$
- Identity: $\mathrm{id}_b \circ f = f = f \circ \mathrm{id}_a$

Categories

Definition (Category)

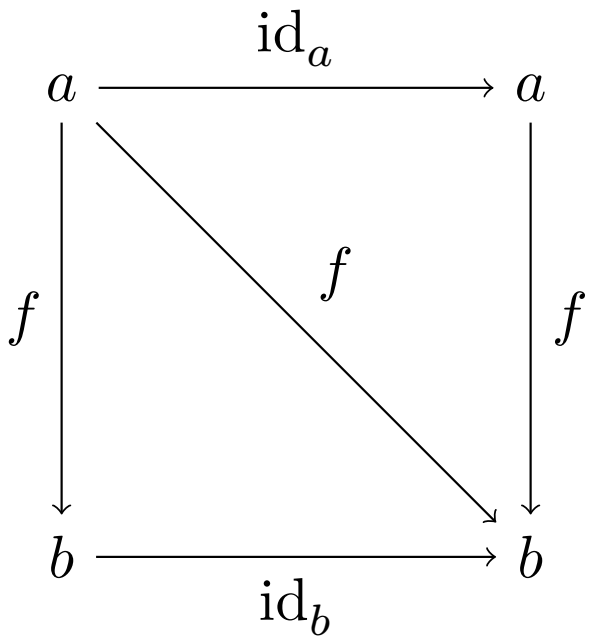
- Associativity: $h \circ (g \circ f) = (h \circ g) \circ f$



Categories

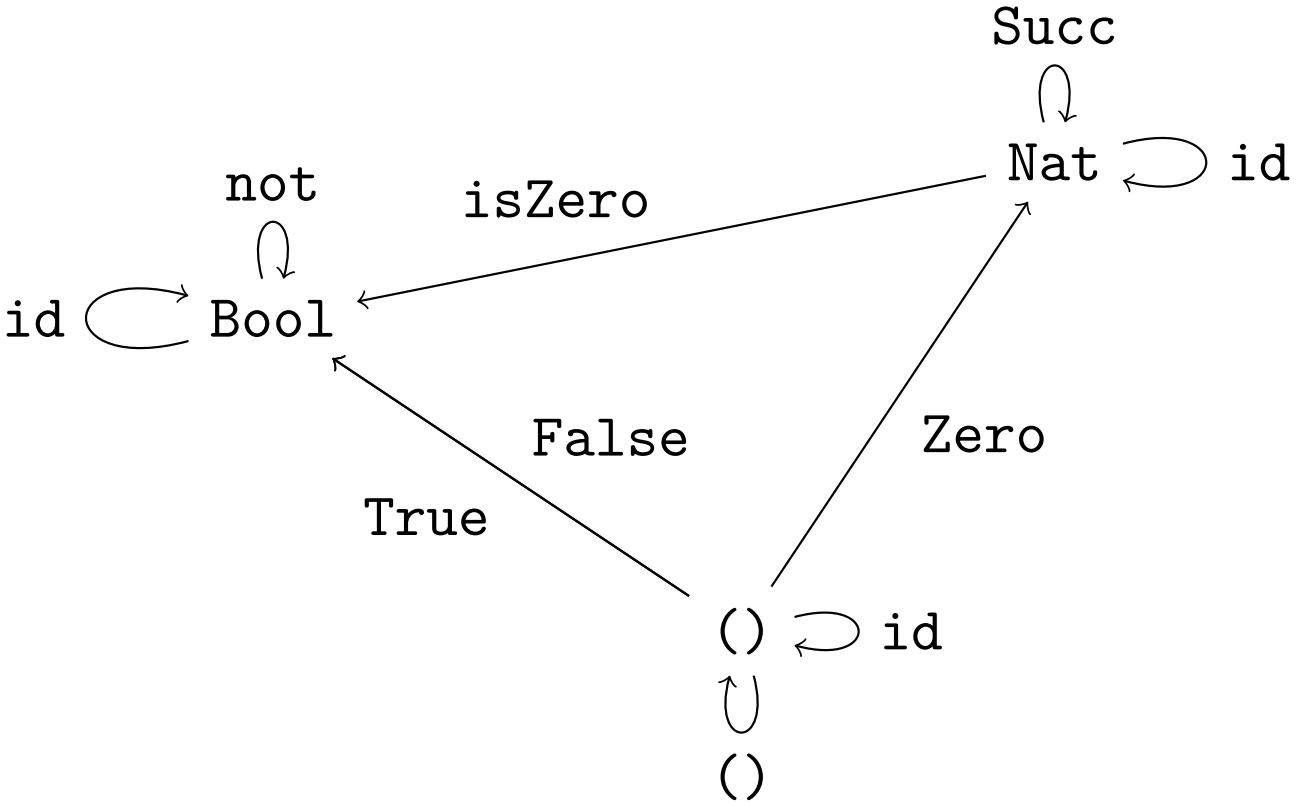
Definition (Category)

- Identity: $\mathrm{id}_b \circ f = f = f \circ \mathrm{id}_a$



Categories

A Category for Haskell



Categories

A Category for Haskell

Example

Hask is the category of Haskell types and functions.

- Objects: Haskell types with kind `*`, like `Bool`:

```
data Bool = False | True
```

- Morphisms: Haskell functions, like `not`:

```
not :: Bool -> Bool -> Bool
not False = True
not True  = False
```

Categories

A Category for Haskell

Example

Hask is the category of Haskell types and functions.

- Identities:

```
id :: a -> a
id x = x
```

- Composites:

```
(.) :: (b -> c) -> (a -> b) -> a -> c
(g . f) x = g (f x)
```

Categories

A Category for Haskell

Bottom

- Haskell types have bottom and do not yield a category.
- By Haskell types, we actually mean Haskell types without bottom, which yield a category: $\mathbf{Hask}$.

▸ Bottom

Functors

Category *has been defined in order to be able to define* functor...

—Saunders Mac Lane

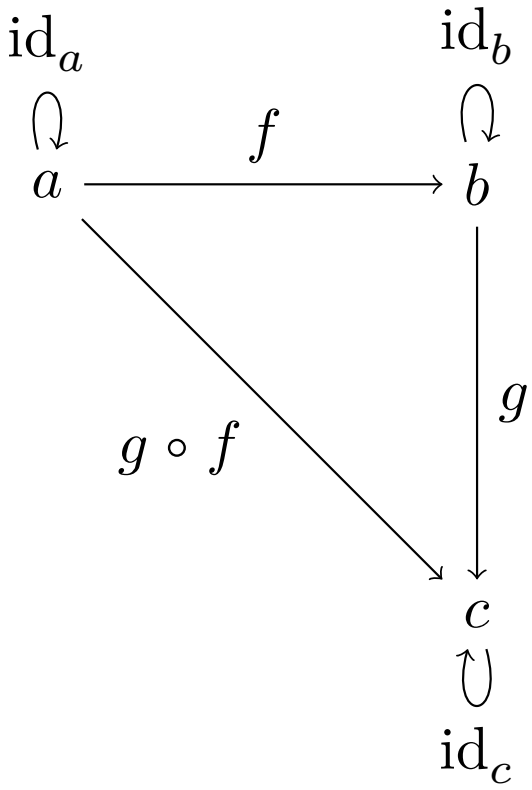
Functors

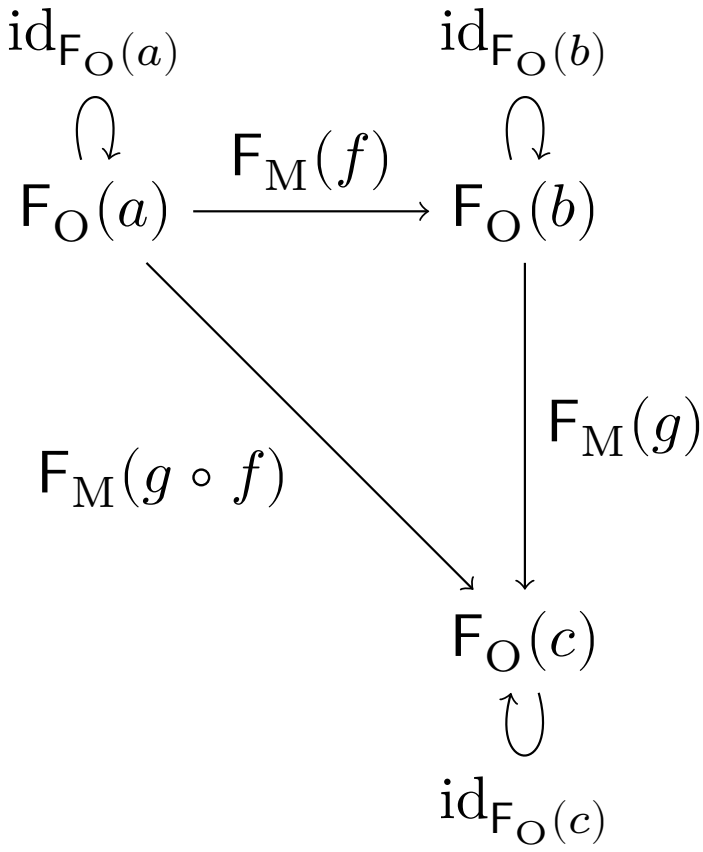
Definition (Functor)

- Let $\mathcal{C}$ and $\mathcal{D}$ be categories.
- A functor $\mathsf{F} : \mathcal{C} \to \mathcal{D}$ assigns:
  - To each object a in $\mathcal{C}$,
    an object $\mathsf{F}_{\mathrm{O}}(a)$ in $\mathcal{D}$.
  - To each morphism $f : a \to b$ in $\mathcal{C}$,
    a morphism $\mathsf{F}_{\mathrm{M}}(f) : \mathsf{F}_{\mathrm{O}}(a) \to \mathsf{F}_{\mathrm{O}}(b)$ in $\mathcal{D}$.
- Such that:
  - $\mathsf{F}_{\mathrm{M}}(\mathrm{id}_a) = \mathrm{id}_{\mathsf{F}_{\mathrm{O}}(a)}$
  - $\mathsf{F}_{\mathrm{M}}(g \circ f) = \mathsf{F}_{\mathrm{M}}(g) \circ \mathsf{F}_{\mathrm{M}}(f)$

Functors

Definition (Functor)





Functors

Definition (Endofunctor)

- Let $\mathcal{C}$ be a category.
- A functor $F : \mathcal{C} \to \mathcal{C}$ is called an endofunctor.

Functors

Functors in Haskell

Definition (`Functor`)

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

Functors

Functors in Haskell

Definition (`Functor`)

A `Functor` is an endofunctor in **Hask**:

```
class Functor (f :: * -> *) where
  fmap :: (a -> b) -> (f a -> f b)
```

Such that:

```
fmap id      = id
fmap (g . f) = fmap g . fmap f
```

Functors

Functors in Haskell

Example

```
data Maybe a = Nothing | Just a

instance Functor Maybe where
  fmap :: (a -> b) -> Maybe a -> Maybe b
  fmap _ Nothing  = Nothing
  fmap f (Just x) = Just (f x)
```

Functors

Functors in Haskell

Example

```
data [] a = [] | a : [a]

instance Functor [] where
  fmap :: (a -> b) -> [a] -> [b]
  fmap _ []     = []
  fmap f (x:xs) = f x : fmap f xs
```

Natural Transformations

... and functor *has been defined in order to be able to define* natural transformation.

—Saunders Mac Lane

Natural Transformations

Definition (Natural transformation)

- Let $\mathcal{C}$ and $\mathcal{D}$ be categories.
- Let F and G : $\mathcal{C} \to \mathcal{D}$ be functors.
- A natural transformation τ : F $\to$ G : $\mathcal{C} \to \mathcal{D}$ assigns:
  - To each object a in $\mathcal{C}$,
    a morphism $\tau_a : \mathsf{F}_{\mathrm{O}}(a) \to \mathsf{G}_{\mathrm{O}}(a)$ in $\mathcal{D}$.
- Such that:
  - Naturality: $\tau_b \circ \mathsf{F}_{\mathrm{M}}(f) = \mathsf{G}_{\mathrm{M}}(f) \circ \tau_a$

Natural Transformations

Definition (Natural transformation)

- Naturality: $\tau_b \circ \mathsf{F}_{\mathrm{M}}(f) = \mathsf{G}_{\mathrm{M}}(f) \circ \tau_a$

$$\begin{array}{ccc} \mathsf{F}_{\mathrm{O}}(a) & \xrightarrow{\tau_a} & \mathsf{G}_{\mathrm{O}}(a) \\ \downarrow{\scriptstyle \mathsf{F}_{\mathrm{M}}(f)} & & \downarrow{\scriptstyle \mathsf{G}_{\mathrm{M}}(f)} \\ \mathsf{F}_{\mathrm{O}}(b) & \xrightarrow[\tau_b]{} & \mathsf{G}_{\mathrm{O}}(b) \end{array}$$

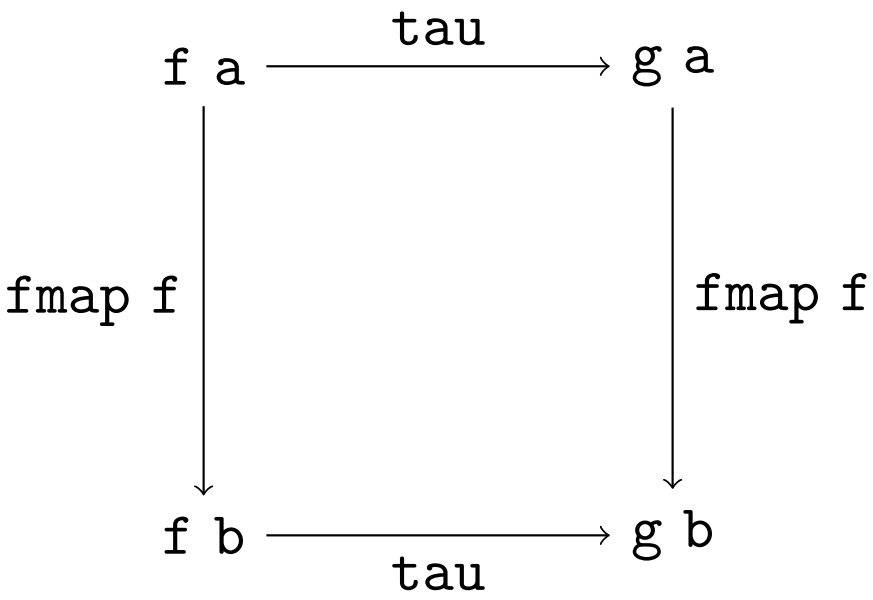
Natural Transformations

Natural Transformations in Haskell

A parametrically polymorphic function is a natural transformation.

Natural Transformations

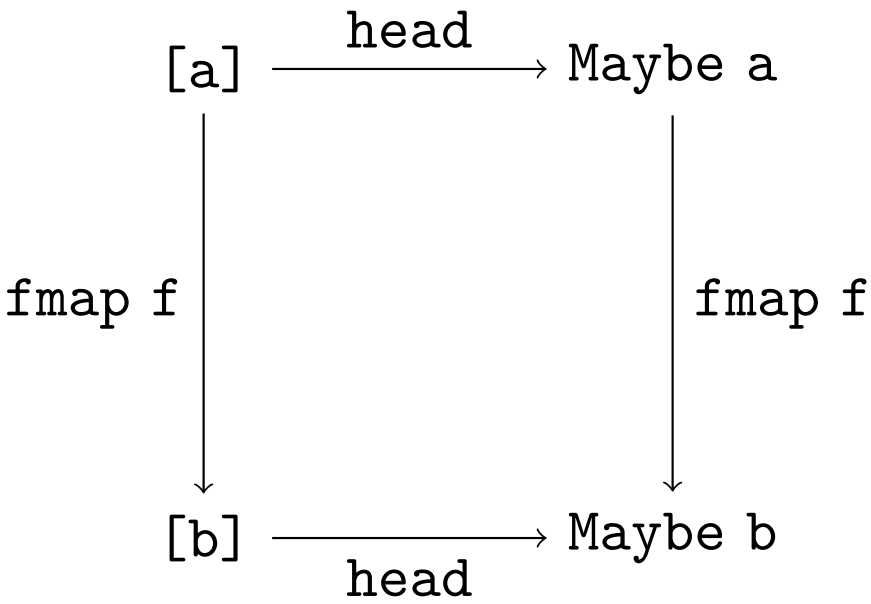
Natural Transformations in Haskell



Natural Transformations

Natural Transformations in Haskell

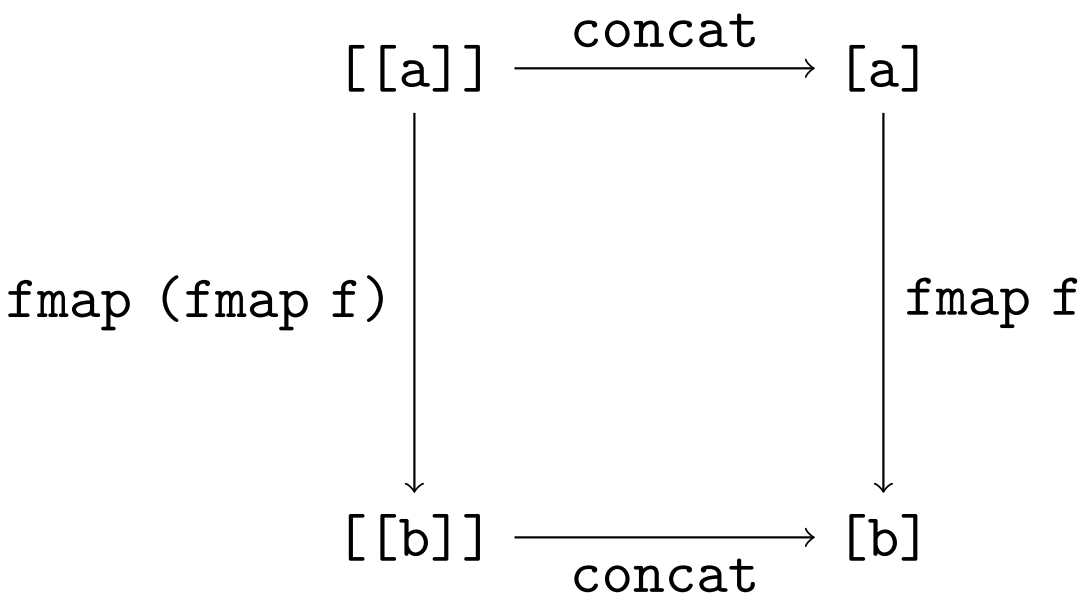
Example



Natural Transformations

Natural Transformations in Haskell

Example



Conclusions

- We did not cover all of category theory.
- But we did cover categories, functors, and natural transformations:
  - An appropriate starting point for the study of category theory applied to functional programming.
  - An overview of the project.

Conclusions

- Studying category theory may not be necessary for functional programming.
- But it is useful for:
  - Better understanding functional programming.
  - Becoming a better functional programmer.

Conclusions

- Difficult? At first.

One does not so much learn category theory as absorb it over a period of time.

—Richard Bird and Oege de Moor

- Definitely worth it.

Conclusions

Future Work

- Adjoints, which "arise everywhere."
- Applicative functors.
  - Monoidal categories and functors.
  - Haskell 2014 `Applicative => Monad` proposal.

Conclusions

Future Work

- Categories.
  - The `Category` type class.
  - The `Arrow` type class.
- Folds.
  - The `Foldable` type class.
  - The `Traversable` type class.
- Monoids, a "fundamental notion of category theory."
  - The `Monoid` type class.

Bibliography

Category Theory


Awodey, Steve (2010).
Category Theory.
2nd ed. Vol. 52. Oxford Logic Guides.
Oxford University Press.


Mac Lane, Saunders (1998).
Categories for the Working Mathematician.
2nd ed. Vol. 5. Graduate Texts in Mathematics.
Springer.

Bibliography

Category Theory and Functional Programming


Bird, Richard and Oege de Moor (1997).
Algebra of Programming.
Prentice Hall.


Pierce, Benjamin C. (1991).
Basic Category Theory for Computer Scientists.
MIT Press.


Poigné, Axel (1992).
Basic Category Theory.
In: Handbook of Logic in Computer Science. Vol. 1.
Clarendon Press.

Bibliography

Category Theory and Haskell


Elkins, Derek (2009).
Calculating Monads with Category Theory.
The Monad.Reader 13, pp. 73–91.


Yorgey, Brent (2009).
The Typeclassopedia.
The Monad.Reader 13, pp. 17–68.

Bibliography

Haskell


Lipovača, Miran (2011).
Learn You a Haskell for Great Good! A Beginner's Guide.
No Starch Press.


O'Sullivan, Bryan, John Goerzen, and Don Stewart (2008).
Real World Haskell.
O'Reilly.

Contents

Categories

Example

Set is the category of sets and functions.

- Objects: Sets A, B, C, ...
- Morphisms: Functions $f : A \to B$, $g : B \to C$, ...
- Identities: $\mathrm{id}_A : A \to A$ such that $\mathrm{id}_A(x) = x$, ...
- Composites: $g \circ f : A \to C$ such that $(g \circ f)(x) = g(f(x))$, ...

Categories

Foundations

- To some extent, we are considering:
  - Objects of $\mathbf{Set}$: The set of all sets.
- This would lead us to a paradox such as Russell's paradox.
- We ought to assume that there is a big enough set, the universe, and take:
  - Objects of $\mathbf{Set}$: The sets which are members of the universe.

Categories

A Category for Haskell

Bottom

- Haskell types have bottom:

```
undefined :: a
undefined = undefined
```

- Haskell types do not yield a category.
- By Haskell types, we actually mean Haskell types without bottom, which yield a category: **Hask**.

See `http://www.haskell.org/haskellwiki/Hask`.

Return

Functors

Functors in Haskell

Example?

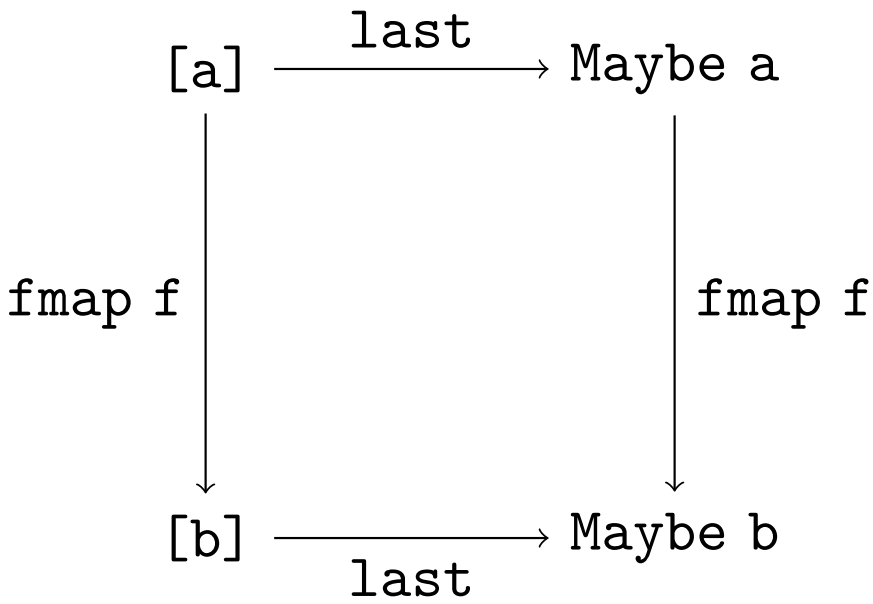
```
data [] a = [] | a : [a]

instance Functor [] where
  fmap :: (a -> b) -> [a] -> [b]
  fmap _ []     = []
  fmap f (x:xs) = f x : f x : fmap f xs
```

Natural Transformations

Natural Transformations in Haskell

Example



Monads

Definition (Monad)

- Let $\mathcal{C}$ be a category.
- A monad $\mathsf{T} = (\mathsf{T}, \eta, \mu)$ in $\mathcal{C}$ consists of:
  - An endofunctor $\mathsf{T} : \mathcal{C} \to \mathcal{C}$.
  - Natural transformations:
    - Unit: $\eta : \mathsf{I} \to \mathsf{T} : \mathcal{C} \to \mathcal{C}$
    - Multiplication: $\mu : \mathsf{T} \circ \mathsf{T} \to \mathsf{T} : \mathcal{C} \to \mathcal{C}$
- Such that:
  - $\mu_a \circ \mu_{\mathsf{T}_{\mathrm{O}}(a)} = \mu_a \circ \mathsf{T}_{\mathrm{M}}(\mu_a)$
  - $\mu_a \circ \eta_{\mathsf{T}_{\mathrm{O}}(a)} = \mathrm{id}_{\mathsf{T}_{\mathrm{O}}(a)} = \mu_a \circ \mathsf{T}_{\mathrm{M}}(\eta_a)$

Monads

Monads in Haskell

Definition (Monad')

```
class Functor m => Monad' m where
  return :: a -> m a
  join   :: m (m a) -> m a
```

Monads

Monads in Haskell

Example

```
instance Monad' Maybe where
  return :: a -> Maybe a
  return = Just

  join :: Maybe (Maybe a) -> Maybe a
  join Nothing   = Nothing
  join (Just mx) = mx
```

Kleisli Triples

Definition (Kleisli triple)

- Let $\mathcal{C}$ be a category.
- A Kleisli triple $\mathsf{T} = (\mathsf{T}_O, \eta, \_^*)$ in $\mathcal{C}$ consists of:
  - A natural transformation:
    - Unit: $\eta : \mathsf{I} \to \mathsf{T} : \mathcal{C} \to \mathcal{C}$

  And assigns:
  - To each object a, an object $\mathsf{T}_O(a)$.
  - To each morphism $f : a \to \mathsf{T}_O(b)$, a morphism $f^* : \mathsf{T}_O(a) \to \mathsf{T}_O(b)$.
- Such that:
  - $g^* \circ f^* = (g^* \circ f)^*$
  - $f^* \circ \eta_a = f$
  - $\eta_a^* = \mathrm{id}_{\mathsf{T}_O(a)}$

Kleisli Triples

Kleisli Triples in Haskell

Definition (Monad)

```
class Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b

  ...
```

Kleisli Triples

Kleisli Triples in Haskell

Definition (Monad'')

```
class Monad'' m where
  return :: a -> m a
  bind   :: (a -> m b) -> m a -> m b
```

Kleisli Triples

Kleisli Triples in Haskell

```
Monad = Monad''
```

```
(>>=) :: Monad'' m => m a -> (a -> m b) -> m b
mx >>= f = bind f mx
```

Kleisli Triples

Kleisli Triples in Haskell

Example

```
instance Monad'' Maybe where
  return :: a -> Maybe a
  return = Just

  bind :: (a -> Maybe b) -> Maybe a -> Maybe b
  bind _ Nothing  = Nothing
  bind f (Just x) = f x
```

Monads and Kleisli Triples

Monads and Kleisli Triples in Haskell

`Monad' = Monad''`

```
bind :: Monad' m => (a -> m b) -> m a -> m b
bind f mx = join (fmap f mx)
```

Monads and Kleisli Triples

Monads and Kleisli Triples in Haskell

`Monad' = Monad''`

```
fmap :: Monad'' m => (a -> b) -> m a -> m b
fmap f mx = bind (return . f) mx

join :: Monad'' m => m (m a) -> m a
join mmx = bind id mmx
```

Algebras and Initial Algebras

- Let $\mathcal{C}$ be a category.
- Let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be an endofunctor.

Definition (Algebra)

- An F-algebra (a, α) is:
  - An object a.
  - A morphism $\alpha : \mathsf{F}_{\mathrm{O}}(a) \to a$.

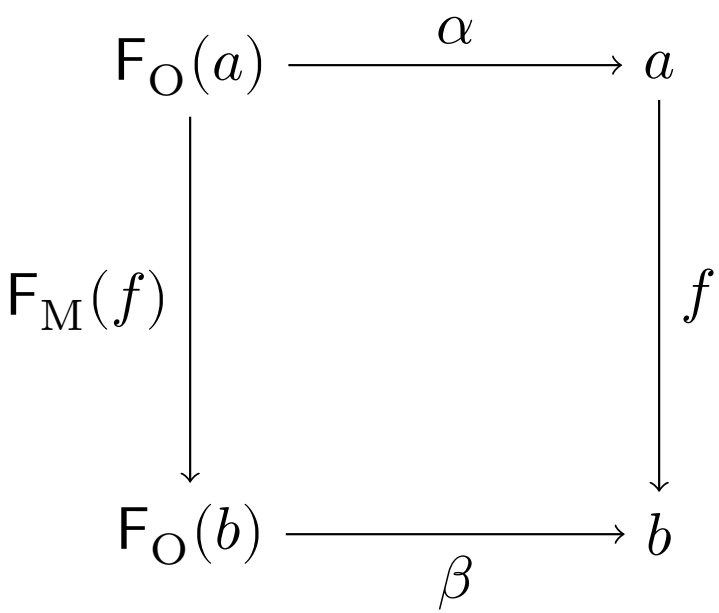
Algebras and Initial Algebras

Definition (Algebra homomorphism)

- Let (a, α) and (b, β) be F-algebras.
- An F-algebra homomorphism $f : (a, \alpha) \to (b, \beta)$ is a morphism $f : a \to b$ in $\mathcal{C}$.
- Such that:
  - $\beta \circ \mathsf{F}_{\mathrm{M}}(f) = f \circ \alpha$

Algebras and Initial Algebras

Definition (Algebra homomorphism)



Algebras and Initial Algebras

Definition

- F-$\mathbf{Alg}$ is the category with:
  - Objects: F-algebras.
  - Morphisms: F-algebra homomorphisms.

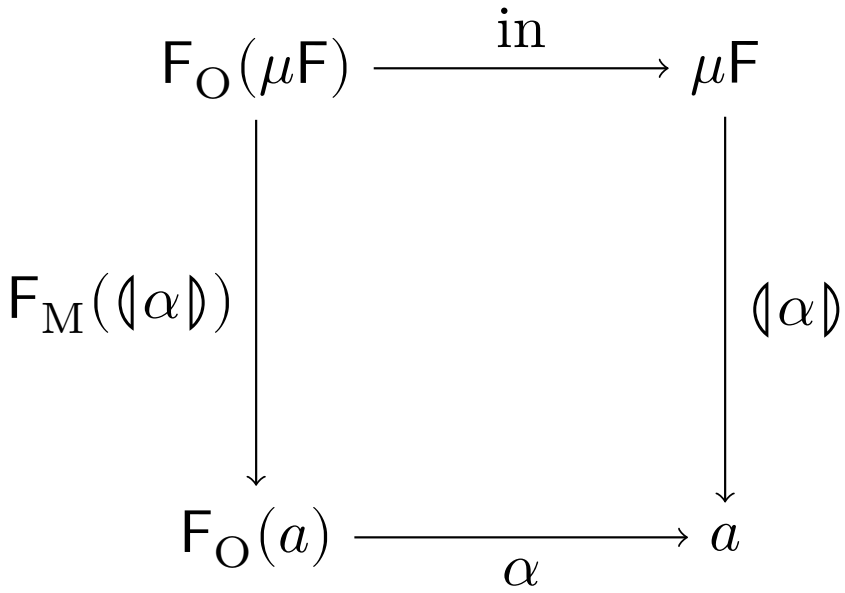
Algebras and Initial Algebras

Definition (Initial algebra)

- An F-algebra $(\mu \mathsf{F}, \mathrm{in})$ is the initial F-algebra of F-$\mathbf{Alg}$ if:
  - For all F-algebras (a, α), there is a unique F-algebra homomorphism $(\!|\alpha|\!) : (\mu \mathsf{F}, \mathrm{in}) \to (a, \alpha)$.
  - That is, a morphism $(\!|\alpha|\!) : \mu \mathsf{F} \to a$ in $\mathcal{C}$ such that:
    - $\alpha \circ \mathsf{F}_{\mathrm{M}}((\!|\alpha|\!)) = (\!|\alpha|\!) \circ \mathrm{in}$

Algebras and Initial Algebras

Definition (Initial algebra)



Algebras and Initial Algebras

Algebras and Initial Algebras in Haskell

Example

```
data [] a = [] | a : [a]
```

This is an L-algebra:

```
data L a b = N | C a b

instance Functor (L a) where
  fmap :: (b -> c) -> L a b -> L a c
  fmap _ N        = N
  fmap g (C x y) = C x (g y)
```

Algebras and Initial Algebras

Algebras and Initial Algebras in Haskell

Example

```
data [] a = [] | a : [a]
```

This is the initial L-algebra:

```
foldr :: b -> (a -> b -> b) -> [a] -> b
foldr n c []     = n
foldr n c (x:xs) = c x (foldr n c xs)
```

Algebras and Initial Algebras

Algebras and Initial Algebras in Haskell

Example

```
(++) :: [a] -> [a] -> [a]
xs ++ ys = (foldr ys (:)) xs
```

```
[]     ++ ys = ys
(x:xs) ++ ys = x : xs ++ ys
```